Как оформлять модели базы данных и SQL-код

Репозиторий с докладами

Владимир Добрышкин

t.me/vodobryshkin

Версия 0.1.1

Аннотация

Данный документ представляет из себя практическую памятку по:

- Нотациям Чена и Crow's Foot.
- Оформлению концептуальной, логической и физической моделей.
- Правилам именования и стилю SQL-кода.

Содержание

Нотации . . . 3
- Нотация Питера Чена . . . 3
- Нотация Crow’s Foot . . . 5

Оформление моделей . . . 7
- Концептуальная модель . . . 8
- Логическая модель . . . 9
- Физическая модель . . . 10

Оформление SQL . . . 11
- Используйте осмысленные названия . . . 11
- Используйте snake_case вместо camelCase . . . 11
- Не используйте венгерскую нотацию . . . 12
- Выделяйте ключевые слова регистром . . . 12
- Разделяйте запрос на логические части . . . 12
- Используйте пробелы и отступы . . . 12
- Используйте понятные псевдонимы и AS . . . 13
- Оформляйте определения таблиц последовательно . . . 13
- Записывайте даты в однозначном формате . . . 14
- Добавляйте содержательные комментарии . . . 14

Обратная связь . . . 14

Источники . . . 14

Нотации

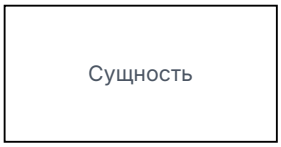
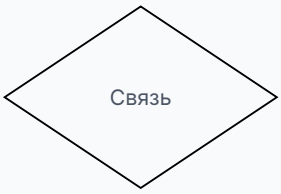
⚠ Перед тем, как говорить об оформлении моделей, нужно познакомиться с нотациями, в которых эти модели описывают.

- Для оформления концептуальной модели могут быть использованы нотация Питера Чена или Crow’s foot.
- Для оформления логической модели используется Crow’s foot.
- Для оформления физической модели используется Crow’s foot.

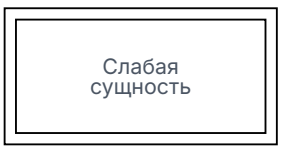
Нотация Питера Чена

ℹ Система графических обозначений для ER-моделей. Она позволяет показать сущности предметной области, их свойства и связи. Питер Чен представил ER-модель и способ её изображения в 1976 году в статье *The Entity-Relationship Model—Toward a Unified View of Data*. Русский перевод можно найти здесь.

Основные компоненты нотации

Компонент	Обозначение	Описание
Сущность		Тип объектов предметной области, о которых нужно хранить данные.
Связь		Ассоциация между сущностями. Может соединять две сущности.
Атрибут		Свойство сущности или связи.

Дополнительные компоненты нотации

Компонент	Обозначение	Описание
Слабая сущность		Для идентификации её экземпляра собственных атрибутов недостаточно: требуется также ключ сущности-владельца.

Компонент	Обозначение	Описание
Идентифицирующая связь		Связь слабой сущности с сущностью-владельцем, участвующая в её идентификации.
Ключевой атрибут		Атрибут, входящий в ключ, который однозначно определяет экземпляр сущности.
Многозначный атрибут		Для одного экземпляра сущности может иметь несколько значений.
Производный атрибут		Атрибут, значение которого вычисляется на основе других данных.

Связи в нотации Чена

Перед рассмотрением обозначений связей определим, что такое кратность связи и обязательность участия в ней.

> **Кратность связи**
>
> Параметр, который задаёт допустимое количество экземпляров одного типа сущности, которые могут быть связаны с одним экземпляром другого типа. Она определяется для каждого направления связи.
>
> Кратность может задаваться минимальным и максимальным значениями: например, 0..1 означает “ноль или один”, а 1..M – “один или несколько”. Минимальное значение показывает, обязательно ли участие в связи, а максимальное – сколько связанных экземпляров допускается.

Тип связи	Обозначение	Как читать
Один к одному (1:1)		Каждому А соответствует не более одного Б, и наоборот. Участие обеих сторон необязательно.
Один ко многим (1:M)		Одному А может соответствовать несколько Б. Каждому Б – не более одного А. Участие обеих сторон необязательно.

Тип связи	Обозначение	Как читать
Многие ко многим (M:N)		Одному А может соответствовать несколько Б, а одному Б – несколько А. Участие обеих сторон необязательно.
1:M. Обязательное участие А и Б		Каждому А соответствует не менее одного Б. Каждый Б связан ровно с одним А.
1:M. Обязательно только А		Каждому А соответствует хотя бы один Б. Каждый Б может быть связан с одним А или не участвовать в связи.
1:M. Обязательно только Б		Каждый А может быть связан с несколькими Б или не участвовать в связи. Каждый Б связан ровно с одним А.

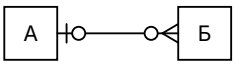
⚠ Часто на ER-диаграммах можно встретить использование только первых трёх видов связи. Чтобы схема точнее передавала правила предметной области, рекомендую указывать не только тип связи – 1:1, 1:M или M:N, – но и обязательность участия каждой стороны.

Нотация Crow's Foot

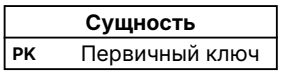
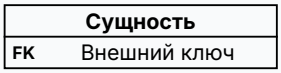
ℹ Система графических обозначений для ER-моделей. Существуют разные соглашения Crow's Foot. Здесь используется вариант, описанный Терри Халпином в статье *Entity Relationship Modeling from an ORM Perspective: Part 3*.

Компоненты нотации

Компонент	Обозначение	Описание
Сущность		Тип объектов предметной области, о которых нужно хранить данные.
Атрибут		Свойство сущности, которое записывается внутри её прямоугольника.

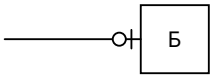
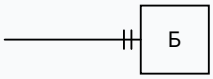
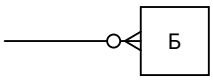
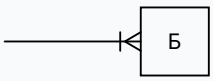
Компонент	Обозначение	Описание
Связь		Ассоциация между сущностями, изображается линией.

Обозначение ключей

Компонент	Обозначение	Описание
Первичный ключ (PK)		Атрибут или набор атрибутов, выбранный для однозначной идентификации экземпляра сущности.
Внешний ключ (FK)		Атрибут или набор атрибутов, ссылающийся на ключ другой или этой же сущности.

Окончания связей в Crow’s Foot

В каждом окончании два символа задают минимум и максимум. Символ ближе к сущности задаёт максимум, второй – минимум.

Кратность	Обозначение	Как читать
Ноль или один (0..1)		Кружок и черта: ноль или один Б.
Ровно один (1..1)		Две черты: ровно один Б.
Ноль, один или несколько (0..M)		Кружок и лапка: ноль, один или несколько Б.
Один или несколько (1..M)		Черта и лапка: один или несколько Б.

> ⚠ Аналогично связям в нотации Чена, чтобы схема правильнее передавала суть предметной области, рекомендуется указывать обязательность участия каждой стороны.

Оформление моделей

После того, как мы разобрались с нотациями, можно наконец описать все три вида моделей. Предлагаю сделать это на примере чуть-чуть видоизменённой предметной области про сдачу экзамена из второй лекции.

Описание предметной области

> ⚠ Описанная предметная область не претендует на полноту по атрибутам. Главная цель – показать, как визуализировать модели.

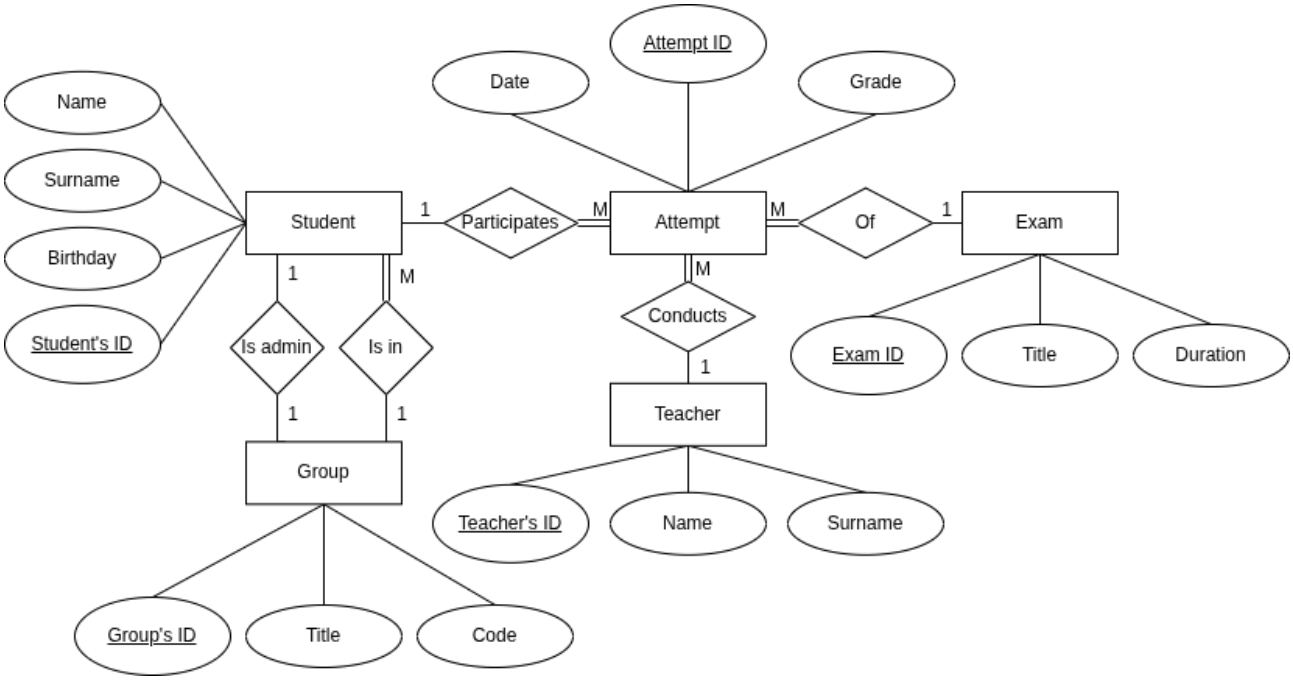
Студент (сущность) имеет имя, фамилию и дату рождения (атрибуты) и состоит ровно в одной учебной группе. Группа (сущность) имеет код, название (атрибуты) и может объединять несколько студентов. Группа может иметь старосту. Преподаватель (сущность) имеет имя и фамилию (атрибуты), экзамен (сущность) – название и длительность (атрибуты). Попытка сдачи (сущность) имеет дату и оценку (атрибуты) и связывает одного студента, один экзамен и одного преподавателя. Студент может повторно сдавать тот же экзамен тому же преподавателю, каждая сдача сохраняется как отдельная попытка.

Инструмент для рисования моделей

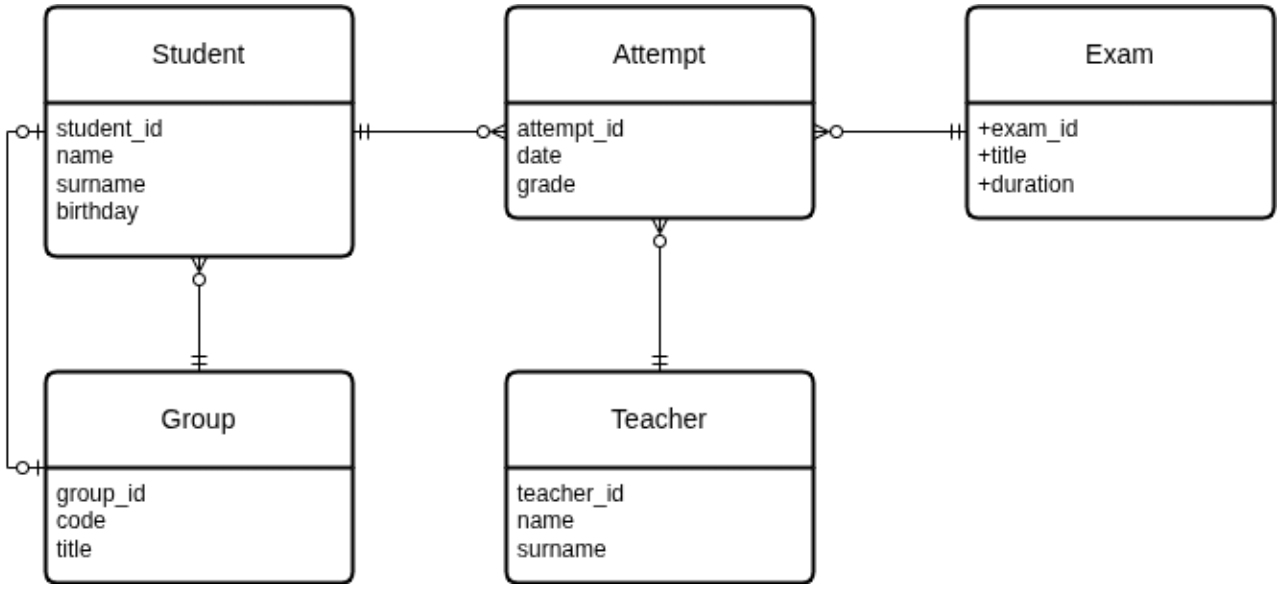
> ℹ Одним из лучших инструментов для рисования моделей является платформа drawio. Она предоставляет графический редактор для различного вида диаграмм с огромными возможностями. Помимо веб-версии, доступна десктопная версия, а так же плагин на IDE от JetBrains (Intellij IDEA, Pycharm, etc.).

Концептуальная модель

> Концептуальная модель нужна для описания предметной области: важных для задачи сущностей, их свойств, связей между ними. Эта модель не зависит от деталей реализации. При концептуальном проектировании мы не задумываемся о будущей модели данных, СУБД, прикладных программах, аппаратной платформе и т.д.



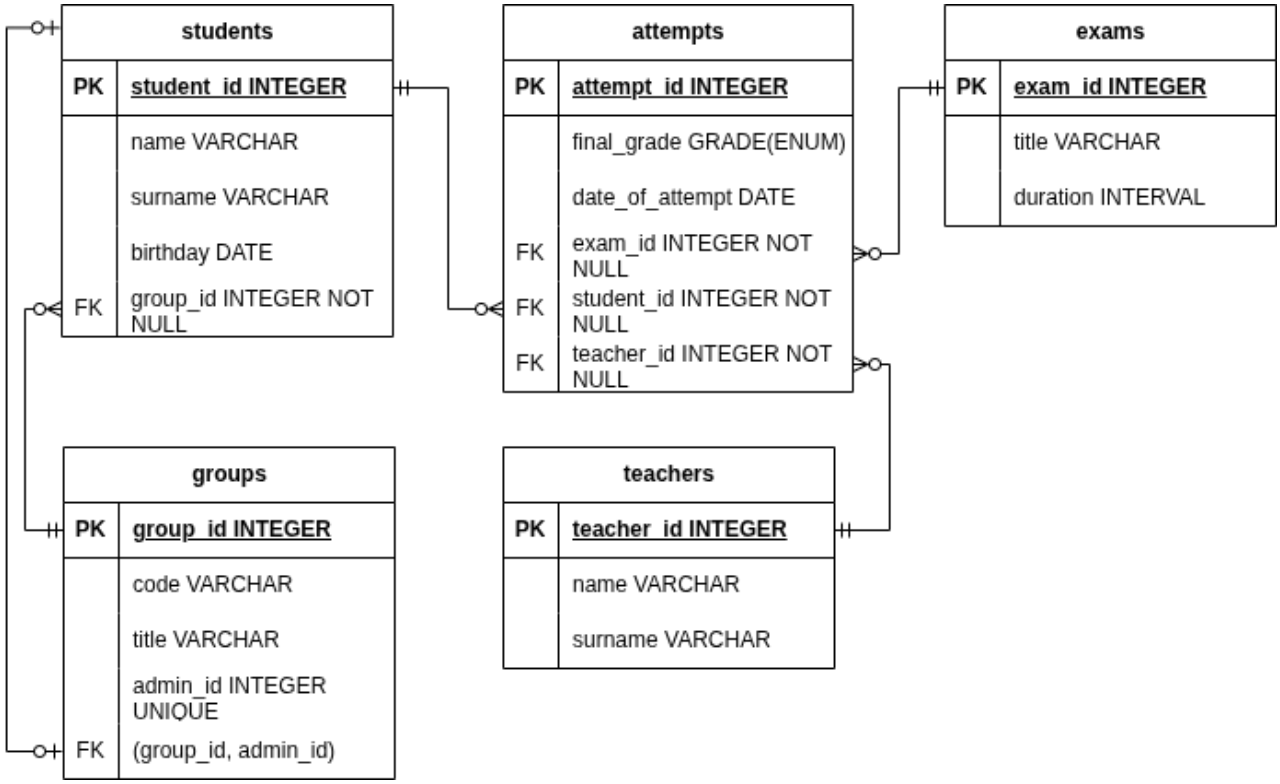
Изображение 1: Концептуальная модель в нотации Питера Чена



Изображение 2: Концептуальная модель в нотации Crow’s foot

Логическая модель

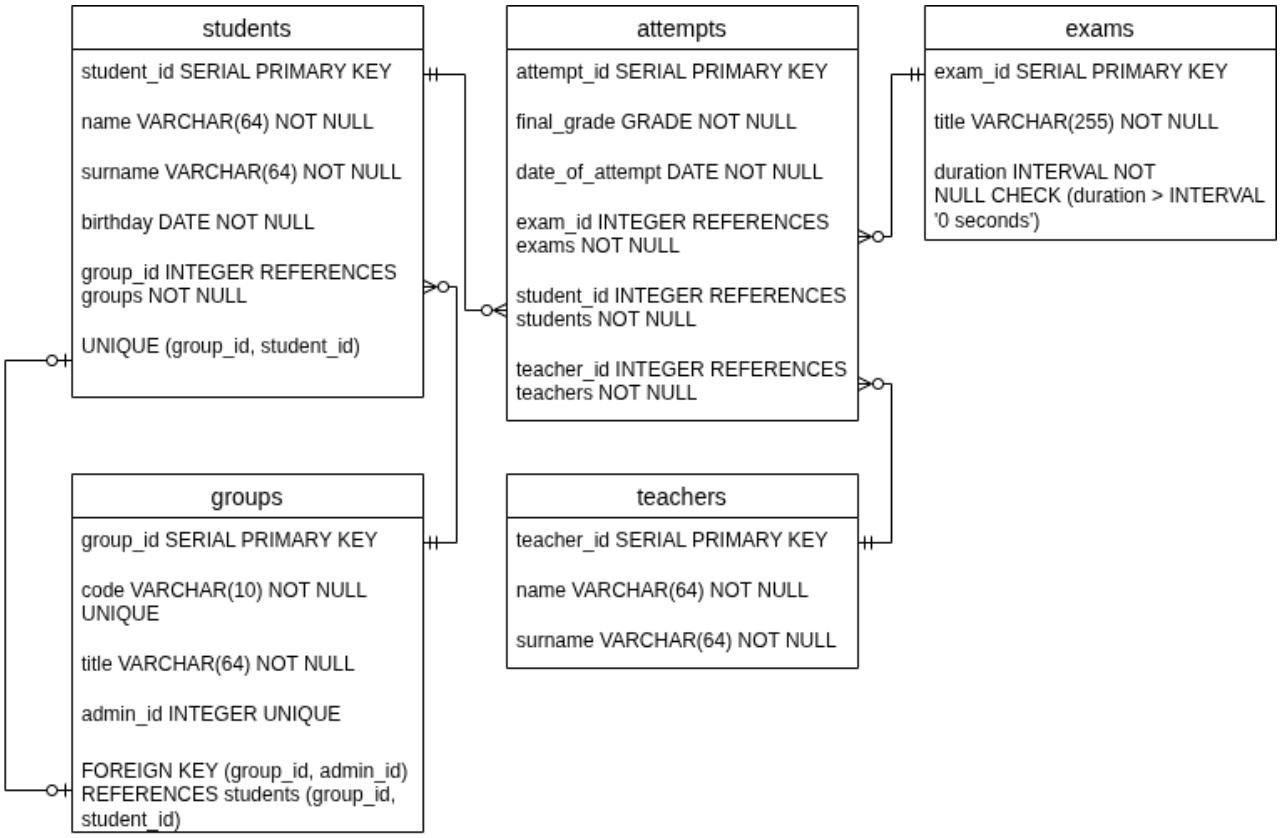
> Логическая модель предназначена для подготовки структуры данных в выбранной модели (например, реляционной), которую затем можно реализовать в конкретной СУБД. Говоря о реляционном логическом проектировании, мы выделям состав данных, атрибуты, ключи, связи и ограничения, но не делаем это в терминах конкретной СУБД.



Изображение 3: Логическая модель

Физическая модель

> Физическая модель описывает, как структура данных будет реализована в конкретной СУБД. При физическом проектировании мы уточняем таблицы и их столбцы, выбираем конкретные типы данных, задаём первичные и внешние ключи, пользовательские ограничения. Цель данного этапа – подготовить проект базы данных, который можно реализовать средствами выбранной СУБД.



Изображение 4: Физическая модель

Оформление SQL

Не существует единого стандарта по оформлению кода на SQL. Правила оформления кода часто отличаются от коллектива к коллективу.

Но есть много хороших SQL Style Guide'ов. Эмпрически, большая часть правил из этих туториалов совпадают с кастомными правилами оформления.

Рассмотрим 10 наиболее полезных (IMHO) правил оформления из одного из самых популярных гайдов – *Руководство по стилю SQL от Саймона Холливелла*.

Примеры этого раздела иллюстрируют стиль SQL и используют самостоятельные условные схемы.

Используйте осмысленные названия

По имени объекта должно быть понятно его содержание. Так вам и вашим коллегам не придётся постоянно сверяться со схемой и догадываться, что означают сокращения.

Плохой пример

```
-- Непонятно, что хранят t, n и d.
SELECT n, d
  FROM t;
```

Хороший пример

```
-- Названия раскрывают содержание:
-- ФИО и даты рождения студентов.
SELECT full_name, birth_date
  FROM students;
```

Используйте snake_case вместо camelCase

Подчёркивания явно разделяют слова и сохраняют эти границы при написании имени строчными буквами. Помимо этого, в PostgreSQL есть дополнительная практическая причина: имена без кавычек приводятся к нижнему регистру. Поэтому при желании сохранить camelCase-названия, вам придется обращаться к ним с двойными кавычками, что неудобно и ухудшает читаемость.

Плохой пример

```
-- Смешанный регистр сохранён.
-- Кавычки нужны при обращении
-- к этим столбцам.
SELECT "fullName", "birthDate"
  FROM students;
```

Хороший пример

```
-- Границы слов видны,
-- кавычки не требуются.
SELECT full_name, birth_date
  FROM students;
```

Не используйте венгерскую нотацию

Префиксы вроде tbl *и str* дублируют информацию, видимую из запроса и определений столбцов.

Плохой пример

```sql
-- Зачем повторять то, что написано
типами?
SELECT str_name, int_group_id
  FROM tbl_students;
```

Хороший пример

```sql
-- Названия отражают смысл.
SELECT name, group_id
  FROM students;
```

Выделяйте ключевые слова регистром

Прописные ключевые слова помогают отличать структуру запроса от названий объектов.

Плохой пример

```sql
-- Ключевые слова визуально
-- не выделяются среди имён.
select full_name
  from students
 where group_id = 10;
```

Хороший пример

```sql
-- Сразу видны выборка,
-- источник данных и фильтр.
SELECT full_name
  FROM students
 WHERE group_id = 10;
```

Разделяйте запрос на логические части

Переносы строк помогают отдельно увидеть выборку, соединения и фильтры. Условия с `AND` или `OR` на отдельных строках проще проверять и изменять.

Плохой пример

```sql
-- Части запроса записаны вместе.
-- Отдельное условие труднее найти.
SELECT name FROM students
WHERE group_id = 10 AND active;
```

Хороший пример

```sql
-- Источник данных и каждое
-- условие находятся отдельно.
SELECT name
  FROM students
 WHERE group_id = 10
   AND active;
```

Используйте пробелы и отступы

Пробелы отделяют имена и выражения друг от друга. Последовательные отступы показывают группировку кода: например, соединение таблиц и относящиеся к нему условия.

Плохой пример

```sql
-- Запятая и = прижаты к именам.
-- Соединение не выделено отступом.
SELECT s.name,g.name
FROM students AS s
JOIN groups AS g
ON s.group_id=g.group_id;
```

Хороший пример

```sql
-- Пробелы разделяют выражения.
-- JOIN и его условие сгруппированы.
SELECT s.name, g.name
  FROM students AS s
       JOIN groups AS g
       ON s.group_id = g.group_id;
```

Используйте понятные псевдонимы и AS

Псевдонимы сокращают длинные обращения, но должны оставаться понятными. Слово AS явно обозначает переименование и явно показывает, что оно есть.

Плохой пример

```
-- q не связано с именем таблицы.
-- x не объясняет, что вычислено.
SELECT AVG(q.mark) x
  FROM exam_results q;
```

Хороший пример

```
-- er сокращает exam_results.
-- Результат назван средней оценкой.
SELECT AVG(er.mark) AS average_mark
  FROM exam_results AS er;
```

Оформляйте определения таблиц последовательно

Отдельные строки позволяют быстро сопоставить столбцы, типы и ограничения. Объявляйте каждый столбец отдельно; внутри CREATE TABLE используйте отступ в четыре пробела.

Плохой пример

```
-- Два столбца слились в строке.
-- Нет отступа внутри определения.
CREATE TABLE students (
student_id INTEGER PRIMARY KEY,
name TEXT NOT NULL, birth_date DATE
);
```

Хороший пример

```
-- Каждый столбец легко найти
-- вместе с его типом и ограничениями.
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    birth_date DATE
);
```

Записывайте даты в однозначном формате

Используйте формат YYYY-MM-DD: по нему понятно, где год, месяц и день. В PostgreSQL такая запись даты не зависит от порядка компонентов, заданного настройкой DateStyle.

Плохой пример

```
-- MDY: 4 марта.
-- DMY: 3 апреля.
-- Настройка меняет смысл записи.
SELECT DATE '03/04/2005';
```

Хороший пример

```
-- Однозначно: 3 апреля 2005 года.
SELECT DATE '2005-04-03';
```

Добавляйте содержательные комментарии

Комментарий полезен, когда объясняет цель запроса или причину неочевидного условия. В это время, пересказ очевидных действий увеличивает объём кода, не помогая понять его назначение.

Плохой пример

```
-- Комментарий пересказывает SELECT,
-- но не объясняет цель запроса.
-- Выбираем student_id.
SELECT student_id
  FROM exam_results
 WHERE mark < 3;
```

Хороший пример

```
-- Комментарий объясняет цель:
-- список студентов на пересдачу.
SELECT student_id
  FROM exam_results
 WHERE mark < 3;
```

Обратная связь

Нашли опечатку или ошибку?

Есть идеи по улучшению материала?

Хотите предложить тему, которой можно будет дополнить доклад?

Пишите в личные сообщения или заведите issue в репозитории!

Источники

1. Моргунов, Е. П. *Введение в теорию баз данных*. Вводная лекция курса “Язык SQL”. Postgres Professional, 2019. Слайды лекции.
2. Чен, П. П.-Ш. *Модель “сущность-связь” — шаг к единому представлению о данных*. Перевод М. Р. Когаловского; новая редакция С. Кузнецова, 2009. Русский перевод на CITForum.

3. Halpin, T. *Entity Relationship Modeling from an ORM Perspective: Part 3*. Статья.
4. IBM. *What Is Data Modeling?* Статья.
5. Microsoft. *Create a Diagram with Crow's Foot Database Notation*. Официальная документация.
6. Holywell, S. *Руководство по стилю SQL*. Русская версия SQL Style Guide.
7. Материалы курса "Базы данных". *Лекция 2: создание Базы Данных*.